

一. 数据结构概念

数据: 信息的载体

- 数值性
- 非数值性

数据结构: Data-Structure = {D, R}

关系的有限集合

数据元素集合

分类

- 线性结构 - 线性表
- 非线性结构
 - 层次结构: 树
 - 网状结构: 集合、图

算法: 一个有穷的指令集

特性: ①有输入 ②有输出 ③确定性 ④有穷性 ⑤可行性 (有效性)

性能标准: ①正确性 ②可使用性 ③可读性 ④效率 ⑤健壮性 ⑥简单性

空间复杂度: 存储空间固定部分: 代码空间、简单变量... 可变部分: 引用变量、动态使用的空间...

时间复杂度:

程序步数

- 注释、声明: 0 函数调用: 0
- 表达式: 1 赋值: 看表达式
- 循环
 - while/do: 内部表达式
 - for: 第一次或1, 2, 之后2, 3
- ...

注: 语句本身的程序步数可能不等于一次执行的程序步数

渐进的时间复杂度: 问题规模 $\rightarrow$ 无穷大, 大O表示法

计算

- 平均循环: 执行频度
- 多个
 - 并列: 加法规则
 - 嵌套: 乘法规则

保守估计 最坏的情况

$$C < \log_2 n < n < n \log_2 n < n^2 < n^3 < 2^n < 3^n < n!$$

大O表示: 乐观估计 (H) 记号: 准确估计

2. 线性表

线性表: 元素有限序列

顺序表: 线性表中的元素相继存放在一个连续的空间中

单链表: 空间不连续

定义方式: 复合、嵌套、结构、继承

插入: 前 $new \rightarrow link = first$ 中: $new \rightarrow link = now \rightarrow link$
 $first = new$ 后: $now \rightarrow link = new$

附加头结点: 简化操作

循环链表: 判空条件 (有附加头结点) $first \rightarrow link == first$

单链指针, 在表尾删除为0, 需找到前一个结点

双向链表: $link, rlink$

静态链表: 为数组元素附加指针 即以下标为索引找下一个

3. 栈与队列

栈: 一端插入 - 端删除 后进后出
top 指向实际栈顶位置

双栈、多栈: 多个栈底/顶指针 栈溢出技术

链式栈: 链表, 灵活

一般地, 设有n个元素按序号1, 2, ..., n进栈, 则可能的出栈有:

$$\frac{1}{n+1} C_n^n \text{ 种}$$

计算表达式

- 后缀: 遇到符号就取前两个数算 (括号归原位)
- 前缀: 从右向左, 遇符号取右边两数算

栈的应用: 递归

递归 $\rightarrow$ 非递归

- 递归算法模拟 (单向递归、尾递归)
- 自己用堆栈模拟

队列: 一端插入另一端删除

数组存储: $front, rear \Rightarrow$ 循环队列, 取模

出 队

队满条件: $(rear+1) \% size == front$

循环队列队满, 实际上空了一个, 为了区分队空条件

链式队列: 无队满问题 带尾指针的循环链表

打印杨辉三角:

```
int i=1, s=0, j, t;
Q.push(i) Q.push(i);
for (i=0; i<n; i++) {
    cout << endl;
    Q.push(0);
    for (j=1; j<=i+2; j++) {
        Q.pop(i);
        Q.push(s+t);
        s=t;
        if (j==i+2) cout << s << " ";
    }
}
```

优先级队列: 取有最高优先权的元素 需调整顺序

双端队列: 两端可进可出

四. 数组、串与广义表

一维数组、多维数组：连续存储

二维数组：

- 行优先存放
- 列优先存放

 } 计算地址
 注意起始/元素大小

三维数组：页/行/列

特殊矩阵：对称矩阵

- 上三角
- 下三角

 → 仅存一半

上三角矩阵：A[i][j] 存于 B[2i+1][j]

0 ≤ i ≤ n-1 第 i+1 行有 3(i+1)-1 个

i-1 ≤ j ≤ i+1 第 i 行有 2 个

本行有 j-2+1 个

稀疏矩阵：三元组表、非行指针数组的三元组表

快速转置算法：利用 rowStart 数组记录每行非零元素在转置三元组表中的存放位置

利用 rowSize 统计原矩阵各列非零元素个数

rowStart[i] = rowStart[i-1] + rowSize[i-1]

主体算法：for (int i = 0; i < terms; i++) {
 "每个元素"
 j = rowStart[elem[i].col];
 "获取对应地址"
 b.elem[j].row = elem[i].col;
 b.elem[j].col = elem[i].row;
 b.elem[j].value = elem[i].value;
 rowStart[elem[i].col]++;
 "地址向右移"
 }

字符串：n 个字符的有限序列

空串和空串不相同 任意连续字符组成子序列：子串

KMP 匹配算法：

```
void generateNext (char *p, int m, int *next) {
    for (int i = 0, j = next[0] = -1; i < m-1; i++) {
        for (; j > -1 && p[i] != p[j]; j = next[j]);
        i++; j++;
        next[i] = j;
    }
}
```

```
void KMP (char *p, int m, char *t, int n) {
    int i, j, next[size]; generateNext (p, m, next);
    for (i = j = 0; j < n; i++) {
        for (; i > -1 && p[i] != t[j]; i = next[i]);
        i++; j++;
        if (i >= m) return j-i;
    }
    return -1;
}
```

广义表：n 个表元素组成的有限序列

↳ 可为子表、可为原子

head: 表的第一个表元素 tail: 除 head 外其他元素组成的表

长度：元素个数 深度：嵌套子表层次

结点定义	atype	info	link
0, 头结点	ref		指向第一个结点
1, 原子	value		
2, 子表	hlink		同层下一结点

递归算法：注意值的更新

五. 树与二叉树

树

- 自由树
- 有根树：根、只有后继

术语：度：结点的子女个数 树的度：结点度的最大值

层次：根在第一层，子女向下+1 } 题中有前提

深度：数值上等于层次
 高度：叶高度为 1，双亲向上+1
 树的高度为根结点高度

二叉树：最多两子女

第 i 层最多有 2^{i-1} 个结点 (i ≥ 1)

深度为 k：最少 k 个 - 最多 $2^k - 1$ 个

叶结点 n_0 、度数为 2 的非叶结点 n_2 ： $n_0 = n_2 + 1$

$n = n_0 + n_1 + n_2$ $2n_2 + n_1 = n - 1$

满二叉树：无缺 完全二叉树：只在最后一层从右向左缺

深度为 k： $2^{k-1} < k \leq 2^k - 1$

二叉链表表示 leftChild data rightChild

遍历

- 中序 (明确结构、必须)
- 前序
- 后序

 访问根结点的顺序不同

层次：队列，根先进队，每次进队将左右子女入队

有 n 个结点的二叉树的形态数： $\frac{1}{n+1} C_{2n}^n$

线索二叉树：l/tag = 0, 为子女 / = 1, 为前/后继

注：当不存在左/右子女时，才为线索

子女-兄弟表示：可构造出二叉树 firstChild nextSibling

树的遍历

- 深度优先：先根、后根
- 广度优先：层次遍历

森林 ⇒ 二叉树：子女-兄弟表示

堆：按完全二叉树存储，有最小/最大堆
 结点值小，结点值大

建立最小堆：(给的序列一般为层次遍历)

- ① 下滑调整：先找到子中较小者，与结点比较，若结点更大，则小者上移，下标下滑，寻找合适位置
- ② 插入：每次插在最后，再上滑调整
- ③ 上滑调整：沿父结点直到根，逐个比较，若小子双亲则上移，直到找到合适位置
- ④ 删除：一般为删去堆顶，之后将最后一个元素填入，下滑调整

Huffman树：取最小权值两个，将和作为新权值加入

$$\text{带权路径长度 WPL} = \sum_{i=1}^n w_i l_i$$

Huffman编码：左0右1，长度最小，无前缀

六. 集合与字典

集合：成员(元素)的一个群集 成员需互不相同
 {可用位(0,1)向量来实现
 建立起元素与整数0,1,2...之间的联系
 用二进制数组(位向量)实现，0表示不在，1表示在
 可用有序链表实现

等价类与并查集

等价类：依据等价关系归并

用链表：有等价对(i,j)，将i链入第j个链表，将j链入第i个链表

并查集：用树的双亲表示存储集合

一个数组中，存有的负数，绝对值表示以此为根的树的结点数
 自然数，该结点的父结点

合并的折衷规则：处理结点j，若j的双亲不是根，则指向根
 再处理j的原双亲，直到原双亲结点双亲为根

△ 散列表：Address = Hash(key)

散列函数：直接定址：线性， $a \times \text{key} + b$
 数学分析

除留余数法：除一个接近总数m的质数p $\text{key} \% p$
 平方取中法
 折叠法

解决冲突的闭散列方法(开地址法)：

线性探查再散列：往后一个寻找，直至有空桶

二次探查再散列：往前、往后 i^2 个寻找，i从1递增

双散列法：利用第二个散列函数 $\text{rehash}(\text{key}) = p$ ，往后p个寻找

开散列方法(链表地址)：每个表元素为一个链表，散列到此的挂在后面

开散列 > 闭散列

除留余数法 > 其他

性能分析：

ASL_{succ}：搜索已有表项的平均探查次数

ASL_{unsucc}：在表中搜索不到已有表项，但能找到插入位置的平均探查次数

$$\text{ASL}_{\text{succ}} = \frac{\text{表中各项成功插入时分别探查了几次}}{\text{表中项数}}$$

$$\text{ASL}_{\text{unsucc}} = \frac{\text{表中各地址若再被散列到，插入新元素需要探查几次}}{\text{表的长度(地址数)}}$$

α 是散列表的装填因子：

$$\alpha = \frac{\text{表中已装有记录数}}{\text{表中预设最大记录数}} = \frac{n}{m}$$

平均搜索长度依赖于 α ，具体如下：

ASL

	成功	不成功
线性探查	$\frac{1}{2}(1 + \frac{1}{1-\alpha})$	$\frac{1}{2}(1 + \frac{1}{1-\alpha^2})$
作随机探查		
二次探查法	$-\frac{1}{2} \ln(1-\alpha)$	$\frac{1}{1-\alpha}$
双散列法		
开散列法	$1 + \frac{\alpha}{2}$	$\alpha + e^{-\alpha} \approx \alpha$

七. 搜索结构

静态搜索表(数组)

顺序搜索：监视哨，下标为size
 递归算法

有序表：可用判定树(内结点为已有，外结点为未查)

折半搜索(二分搜索)： $O(\log_2 n)$

```

while (low <= high) {
    mid = (low + high) / 2;
    if (A[mid] < k) low = mid + 1;
    else if > high = mid - 1;
    else return mid;
}
    
```

二叉搜索树：左子树都小，右子树都大
 (也称二叉排序树，中序下为升序)

树上的搜索：从根结点开始，逐层比较的过程

插入：先搜索

删除：① 叶，直接删 ② 左/右空，用右/左顶替
 ③ 左右都不空，用右中序下第一个代替

加入外结点：扩充二叉搜索树(判定树)

ASL：概率 * 比较次数(层次数)

最坏情况： $O(n)$ 查找长度与结点数、生成过程有关

AVL树: 高度平衡的二叉搜索树

左右子树高度差不超过1

用平衡因子bf表示(右高-左高), 只取-1, 0, 1

平衡化旋转:



删除: 只有一个子树, 用此代替
有两个子树, 用中序下直接前驱代替 } 平衡化 $O(\log n)$

插入: 通过序列表建树, 平衡化

八. 图

定义: 顶点集合 + 顶点间的关系集合

度: 与顶点关联的边数 (有向中入度+出度)

连通分量: 非连通图的极大连通子图

生成树: 连通图的极小连通子图 (n 个顶点, $n-1$ 条边)

完全图 { 无向 $\frac{n(n-1)}{2}$ 条边
有向 $n(n-1)$ 条边

邻接矩阵: $A[i][j] = \begin{cases} 1, & i, j \text{ 相邻 (有向则 } i \rightarrow j) \\ 0, & i, j \text{ 间无边} \end{cases}$
第i行1的个数为出度; 第j列1的个数为入度

邻接表:

A	0 A → 1 → 3
B	1 B → 2
C	2 C → 1
D	3 D → 0

有向图: 分出边表、入边表

带权图: 多一个数据域存权值

邻接多重表: 边结构 mark 顶点1 顶点2 1的下边 2的下边
点结构 data 相邻的第一条边

图的遍历:

① 深度优先 DFS (邻接矩阵)

```
void DFS (graph, visited, node) {  
    visited[node] = true;  
    cout << node << " ";  
    for (int i = 0; i < n; i++) {  
        if (graph[node][i] == 1 && !visited[i]) {  
            DFS (graph, visited, i)  
        }  
    }  
}
```

② 广度优先 BFS

```
visited[curr] = true; cout << curr << " ";  
q.push (node); // 入队  
while (!q.empty()) {  
    int cur = q.front(); q.pop();  
    for -- 遍历各个顶点, 选相邻且未被访问的入队
```

构建最小生成树

① Kruskal 算法 — 稀疏图

找权值最小边 $\Rightarrow$ 加入生成中的图 { 有回路, 删去, 继续
无回路, 保留, 继续
利用并查集、最小堆

② Prim 算法 — 稠密图

从某一顶点出发, 加入U; 取一端在U另一端在V-U的权值最小边加入, 将另一端加入U; 重复
最小堆 + 辅助数组

③ 破圈法

去最大权边, 不连通则加回, 直到只有 $n-1$ 条边

④ Dijkstra 算法 { 若两端不在同一个连通分量, 加入
若在一个, 把环上权值最大边删去

最短路径:

① 边权非负 单源 — Dijkstra 算法

初始化 $fv[] \rightarrow S$ $edges[i][j] \rightarrow dist[i][j]$

找 $\min\{dist[i][j]\}$ (设k时), $S = S \cup \{k\}$

$dist[i][j] = \min\{dist[i][j], dist[i][k] + edge[k][j]\}$

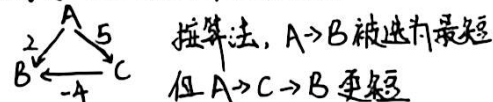
$S = V$, 结束; 否则循环

```
for (int i = 0; i < n; i++) {  
    dist[i] = G.getweight(v, i);  
    sc[i] = false;  
    if (i != v && dist[i] < Maxweight) path[i] = v;  
    else path[i] = -1;  
}
```

```
sc[v] = true; dist[v] = 0;  
for (int i = 0; i < n-1; i++) {  
    min = Maxweight; int u = v;  
    for (int j = 0; j < n; j++)
```

```
        if (sc[j] == false && dist[j] < min) {  
            u = j; min = dist[j];  
        }  
    sc[u] = true;  
    for (int k = 0; k < n; k++) {  
        w = G.getweight(u, k);  
        if (sc[k] == false && w < Max && dist[u] + w < dist[k]) {  
            dist[k] = dist[u] + w;  
            path[k] = u;  
        }  
    }  
}
```

如果存在负权, 直接得到的路可能非最短, 如



② 边上任意值单源路径

B-F方法: 不包含带负权值的回路

源点依次经过其他顶点, 以最短到达终点的最短路径长度

$dist^k[u]$ 是从源点到终点 v 最多经过 k 条边到达的最短路径长度

$$dist^k[u] = \min \{ dist^{k-1}[u], \min \{ dist^{k-1}[j] + edge[j][u], j \text{ 取所有顶点} \} \}$$

③ 所有顶点之间的最短路径

Floyd算法: 定义一个 n 阶方阵序列 $A^{-1} A^0 A^1 \dots A^{n-1}$

$$A^{-1}[i][j] = edge[i][j]$$

$$A^k[i][j] = \min \{ A^{k-1}[i][j], A^{k-1}[i][k] + A^{k-1}[k][j] \}$$

$A^k[i][j]$ 是从 i 到 j , 中间顶点序号不大于 k 的最短路径长度

AOV网络 (顶点表示活动)

拓扑排序: 手工, 选无直接前驱, 删发出边

邻接表中增设数组 $count[i]$, 记录入度

建立入度为0的顶点栈

栈不空时重复:

从栈中取出一个顶点, 输出

从AOV网络中删去, 更新 $count$

若有点入度为0, 入栈

若输出顶点数少, 则报告有环

AOE网络 (边表示活动)

权值 - 持续时间

最长路径长度: 关键路径, 上的所有活动都是关键活动

↓
不按期完成就会影响

九. 排序	最好	最坏
直接插入排序	$O(n)$	$O(n^2)$
折半插入排序	$O(n \cdot \log_2 n)$	
希尔排序	$n^{1.25} \sim 1.6n^{1.25}$	用 gap
冒泡排序	$O(n)$	$O(n^2)$
快速排序 最适用	$O(n \cdot \log_2 n)$ (平均)	$O(n^2)$
选择排序	$O(n^2)$	
堆排序	$O(n \log_2 n)$	
归并排序	$O(n \log_2 n)$	
基数排序	"分配" "收集"	

最好/坏情况下相同:

选择、归并、堆、基数